

RollArt: 通过解耦式基础设施扩展 Agentic RL 训练

Wei Gao^{†*}, Yuheng Zhao^{†*}, Tianyuan Wu^{†*}, Shaopan Xiong^{‡*}, Weixun Wang^{‡*}, Dakai An[‡],
Lunxi Cao[‡], Dilxat Muhtar[‡], Zichen Liu[‡], Haizhou Zhao[‡], Ju Huang[‡], Siran Yang[‡],
Yongbin Li[¶], Wenbo Su[‡], Jiamang Wang[‡], Lin Qu[‡], Bo Zheng[‡], Wei Wang[‡]

[†] 香港科技大学 [‡] 阿里巴巴集团 [¶] 阿里巴巴通义实验室

Abstract

Agentic Reinforcement Learning (RL) 使大语言模型 (LLM) 能够执行自主决策与长程规划。与传统 LLM 后训练不同, agentic RL 的工作负载高度异构, 同时包含计算密集的 prefill、带宽受限的 decoding, 以及具备状态、且严重依赖 CPU 的环境模拟。我们认为, 要高效训练 agentic RL, 必须采用解耦式基础设施, 以便针对不同子任务使用最合适的硬件; 但若直接解耦, 又会因为各阶段间依赖复杂而带来显著的同步开销和资源闲置。

本文提出 RollArt, 这是一个面向解耦式基础设施、用于最大化多任务 agentic RL 吞吐的分布式系统。RollArt 建立在三个核心原则之上: 1) 硬件亲和的工作负载映射, 将计算受限与带宽受限任务路由到最匹配的 GPU; 2) 细粒度异步, 以轨迹为粒度组织执行, 减少流水线气泡; 3) 状态感知计算, 把无状态组件 (如奖励模型) 卸载到无服务器平台以获得弹性伸缩。实验表明, RollArt 能够显著提升训练吞吐, 并相较单体式和同步式基线将端到端训练时间缩短 1.35–2.05 倍。我们还在阿里巴巴超过 3000 张 GPU 的集群上, 用其训练了一个百亿级以上参数规模的 MoE 模型用于 Qoder 产品, 进一步验证了系统的可扩展性与鲁棒性。代码地址: <https://github.com/alibaba/ROLL>。

1 引言

强化学习 (RL) 正在成为推动大语言模型 (LLM) 从被动式逻辑推理迈向自主决策与长时程规划的关键技术 [8, 39, 1]。这一范式通常被称为 *agentic RL*: 模型不再只对单轮输入做响应, 而是要在复杂、动态的环境中持续行动, 场景覆盖工具调用 [15, 53]、网页导航 [48, 24, 22] 以及通用计算机控制 [32, 30, 29]。与传统 LLM 后训练相比, agentic RL 更强调通过大规模“实践”学习: 智能体与外部环境反复交互, 产生长、多轮轨迹, 并在试错中学习解决复杂任务。

agentic RL 训练通常由三个阶段迭代组成: *rollout*、*reward* 与 *training*。在 rollout 阶段, 智能体 LLM 与环境进行多轮反馈式交互, 生成动作 token, 接收观测, 直到满足终止条件, 从而形成一条轨迹。随后在 reward 阶段, 系统利用独立模型或代码沙箱对轨迹进行评估, 并赋予标量奖励。最后, training 阶段消费这些带标签的轨迹, 对智能体参数进行更新, 再将新参数同步回 rollout 侧, 以进入下一轮数据生成与策略更新。

随着研究界开始系统性讨论 agentic RL 的扩展规律 [51, 9], 一个根本性的基础设施问题浮现出来: 整个训练流水线的资源需求不仅高度异构, 而且常常彼此冲突。仅 rollout 阶段就由多种不同负载叠加而成。LLM 生成会在需要高 TFLOPS 的 prefill 阶段与受显存带宽限制的 decoding 阶段之间切换, 前者更适合 H800 等算力型 GPU, 后者则更受益于 H20 等高带宽设备。具体哪个阶段占主导, 又取决于任务形态: 如 FrozenLake [10] 与 SWE-bench [23] 这类长程任务更偏 prefill, 而 GEM-math/GEM-game [4] 这类短轮次任务更偏 decoding。同时, 智能体还需要与成千上万个并行环境交互; 这些环境往往是有状态、CPU 受限的模拟程序, 若与 GPU 推理节点共置, 会引入严重资源争用。相比之下, reward 阶段通常由无状态且可并行的工作者组成, 其计算形式从简单的 CPU 规则脚本 [18, 12] 到复杂的 GPU 评判模型 [47, 59] 都可能出现。

这种异构性意味着, 单体式部署很难高效利用资源。近来的 RL 系统已经开始尝试按阶段做物理解耦, 例如把训练放在计算型 GPU 上, 把 rollout 放到更经济的推理型 GPU 上。然而我们发现, 对于 agentic RL, 仅做阶段级拆分仍然不够: 在 rollout 内部, 不同任务甚至不同轨迹都可能呈现完全不同的硬件亲和性; 在环境侧, 长尾延迟与失败恢复又会让同步式执行大量浪费 GPU 时间; 在奖励侧, 无状态组件更适合迁移到可弹性伸缩的外部服务上, 而不是长期占用热备 GPU。

基于这些观察, 我们提出三项设计原则。第一, **硬件亲和和映射**: 不仅允许用户把整个阶段绑定到特定硬件池, 还允许在阶段内部按更细粒度把子任务分配到最合适的设备上。第二, **细粒度异步**: 在 rollout 内以轨迹而非批次作为调度单位, 并对 rollout 与 training 之间的异步程度进行显式管理, 以平衡吞吐、稳定性与同步成本。第三, **状态感知计算**: 系统应显式区分有状态与无状态组件, 将奖励模型等天然无状态部分迁移到无服务器基础设施, 以获得自动扩缩容、多租户复用和更低的空闲成本。

我们将这些原则落实为 RollArt: 一个结合声明式编程模型与异构感知运行时的分布式系统。用户通过 Python 装饰器声明任务的硬件亲和性, 并将无状态组件注册到外部 serverless 接口。系统内部的资源管理器负责分配异构资源到不同 worker, 分布式运行时则以轨迹粒度编排异步工作流。为充分发挥硬件能力, RollArt 接入了高吞吐推理引擎 [25, 40, 2] 与训练引擎 [46], 并暴露多种传输通道, 以利用 NVLink、InfiniBand 与以太网等不同网络结构, 在轨迹传输与权重同步之间做最优匹配。

我们从零实现了 RollArt, 总代码约 6 万行 Python。基于 Qwen 系列模型 [7] 的实验表明, 它在解耦式集群

*Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong 与 Weixun Wang 对本文贡献相同。

Table 1: 本文采用的 agentic 环境分类。

环境	任务领域	模态	轮次数
SWE-bench [23]	软件工程	文本	30-50
WebShop [55]	Web	文本	5-30
FrozenLake [10]	游戏	文本、视觉	20-100
GEM-math [4]	数学 + 工具调用	文本	< 5
GEM-game [4]	游戏	文本	1

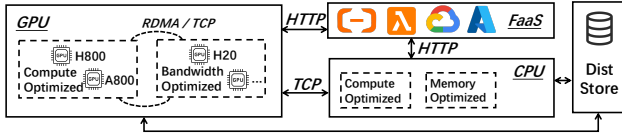


Figure 1: 面向 agentic RL 训练的解耦式基础设施。

上相较单体式和同步式基线最高可获得 2.05 倍加速。为了验证在真实生产中的可扩展性与稳定性，我们还用 RollArt 为 Qoder [37] 训练了大规模 MoE 模型，并在持续一周、超过 3000 张 GPU 的部署中验证了其高吞吐与高鲁棒性。

2 背景

2.1 Agentic RL 训练

训练流水线。多任务 agentic RL 的训练流程由三个计算阶段构成循环。首先是用于采样经验的 **rollout** 阶段：智能体 LLM 与多个并行环境交互，生成训练数据。与普通 LLM 推理不同，这一过程具备多轮与有状态特征 [51, 9]。在每一轮里，智能体观察状态、生成动作序列并提交给环境，环境执行动作后返回反馈，直到满足终止条件，形成由状态-动作对组成的轨迹。随后进入 **reward** 阶段，奖励工作者对轨迹质量进行评估，从轻量级规则打分 [18] 到基于模型的裁判式判断 [47, 59] 都属于这一阶段。最后，**training** 阶段利用这些轨迹和奖励通过 PPO、GRPO 等算法更新参数 [38, 43]。出于训练性能考虑，很多 RL 系统仍然采用同步范式，即要求 rollout 与 training 在每一步严格对齐模型权重 [11]。

环境异构性。agentic RL 的一个关键挑战是环境极其多样，而这种多样性直接决定了系统计算画像。Table 1 显示，不同任务在模态、交互轮次以及环境复杂度上差异极大。长回合任务需要频繁维护上下文与环境状态，往往由 **prefill** 代价主导；而短回合任务更多体现为高频 **decoding**。环境本身还可能依赖浏览器、Docker、代码沙箱、数据库或外部网络服务，因此其瓶颈可能来自 CPU、磁盘、网络或容器生命周期管理，而非 GPU。

为何需要解耦。这种异构性决定了“一个资源池包打天下”的方式效率很低。理想情况下，训练应运行在高算力 GPU 上，环境应运行在 CPU 集群上，奖励模型则可以部署在独立 GPU 或 serverless 平台中。更进一步，在 rollout 内部，不同任务也应被分配到不同 GPU。例如 **prefill** 主导任务更适合 H800，而 **decode** 主导任务可能更适合 H20。如果所有 rollout 都绑定到同一种 GPU，系统将难以充分利用可用硬件。

同步训练与异步训练。同步 RL 训练实现简单，也有利于控制样本新鲜度，但在 agentic RL 中代价很高。环境的长尾延迟、失败重试、不同任务的轮次差异都会让整批样本被最慢的少数轨迹拖住。异步训练允许 rollout 与 training 并行推进，训练端可以在一定范围内

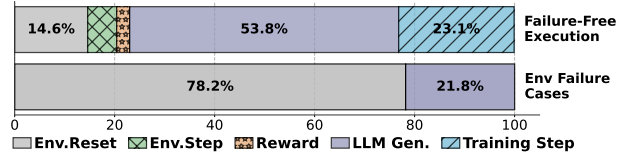
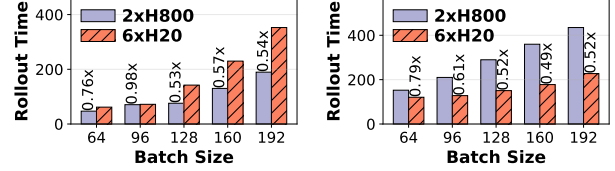


Figure 2: 训练步耗时分解：上图为成功执行，下图为伴随环境失败的执行。



(a) FrozenLake (prefill 主导)。 (b) GEM-Math (decoding 主导)。

Figure 3: 在 H20 与 H800 上，不同任务随 batch size 变化的 rollout 总时长。

消费略旧的样本，以换取更高吞吐。但异步也引入样本陈旧性与稳定性风险，因此需要一套精细的控制机制来限制旧轨迹被使用的程度。

单步训练延迟拆解。我们在 SWE-bench 上分析一步训练的耗时。环境初始化主要由 Docker 镜像拉取、容器启动和沙箱准备构成；环境执行则受代码运行和交互逻辑影响；LLM 生成主导大部分 GPU 时间。实验显示，在完全成功的迭代里，生成阶段是主要开销；但在包含环境失败的迭代里，**env.reset** 会迅速成为瓶颈，平均步时从约 366 秒增加到 513 秒，其中仅环境初始化就可占 rollout 时间的 78%。这说明在 agentic RL 中，环境生命周期管理不是边缘问题，而是核心系统问题。

生成阶段存在分化的硬件亲和性。现代 GPU 在算力、显存容量、带宽与成本上存在不同权衡。已有工作倡导将 rollout 与 training 分置到不同 GPU 上 [58, 14, 52]，但我们的测量表明，对 **agentic** 工作负载而言，这种静态划分仍不够细。LLM 生成同时包含 **prefill** 与 **decoding** 两个资源特征完全不同的阶段；同一个 rollout 集群里，不同任务可能对不同 GPU 呈现截然不同的偏好。Figure 3 就显示，FrozenLake 更适合算力型设备，而 GEM-Math 在带宽型设备上更高效。

长尾环境执行。真实环境存在明显长尾分布。浏览器环境可能因页面加载而阻塞，代码环境可能因容器初始化失败而超时，游戏环境则可能出现异常重置。若系统坚持批次同步，就必须等待最慢环境完成，从而把大量 GPU 时间浪费在空转上。更合理的方式是按轨迹管理环境状态，让慢环境不阻塞快环境，并允许系统主动终止那些超出容忍窗口的执行。

无状态奖励计算。奖励计算通常只依赖输入轨迹，不依赖历史状态，这使其天然适合 **serverless** 平台：按需启动、自动扩缩容、多租户共享、失败隔离都更容易实现。与其为奖励模型常驻预留 GPU，不如把它变成一种共享式的 **Reward-as-a-Service**。

稳定性优先的轨迹传输。轨迹数据本身相对权重同步来说并不大，但它出现在高频环境交互路径中。因此，轨迹传输更强调网络稳定性和低抖动，而不只是峰值带宽。通过把环境交互与 LLM 生成异步化，系统可以隐藏网络波动带来的停顿。

带宽密集的权重更新。与轨迹不同，模型权重同步是真正的带宽密集型通信。它需要高吞吐传输引擎与跨集群优化，否则会吞噬异步训练带来的收益。这一观察意味着系统必须为不同类型的数据流使用不同的传

输策略，而不是统一采用同一套通信机制。

3 设计原则

为满足多任务 agentic 训练在解耦式基础设施上的需求，我们提出以下三项设计原则。

3.1 P1：基于硬件亲和性的工作负载映射

第一项原则是根据硬件亲和性把工作负载映射到资源上，以满足 **R1**。用户不仅可以在阶段粒度定义资源类型集合，例如让训练跑在 GPU、环境跑在 CPU，还可以在轨迹粒度上进行更细的调度。例如在 rollout 阶段，将 prefill 主导、且未触及显存瓶颈的轨迹路由到 H800，而把 decoding 主导的轨迹安排到 H20。奖励阶段同样可以细分：奖励 LLM 跑在 GPU，而代码沙箱运行在 CPU。这样的细粒度映射能够更充分地榨取异构硬件效率。

3.2 P2：细粒度异步执行

第二项原则针对 **R2** 与 **R4**，即利用细粒度异步来应对解耦式 agentic 训练中的多样化通信模式。

轨迹级 rollout 调度。传统方法 [45] 通常把 LLM 生成、环境交互和奖励计算按批次串行组织，这会造成明显资源浪费与高 rollout 延迟。与之相对，轨迹级调度把三者编排成持续流动的流水线：一条轨迹在做环境交互时，另一条轨迹可以进行 LLM 生成，第三条轨迹则进行奖励计算。这样既能减少资源空闲，又能隐藏环境通信延迟，并提升系统对环境不稳定性的耐受能力。

受控的 rollout-train 同步。异步 RL 训练允许 rollout 与 training 并行推进。rollout 持续产出轨迹，training 消费轨迹并周期性地把新权重同步给推理工作者。RollArt 将“已完成轨迹的消费速率”作为显式控制对象，让实践者可以通过异步边界来在训练稳定性、系统吞吐以及权重同步开销之间进行平衡。

3.3 P3：状态感知计算

第三项原则满足 **R3**：系统按组件的状态属性组织计算。无状态组件的输出仅由输入决定，天然适合在 serverless 平台上优化。

Rollout：LLM 生成。生产系统通常使用 serverful 架构部署 LLM 生成，以获得更低时延与更高吞吐。近期工作如 Tinker [26] 开始研究 RL rollout 的弹性服务，但其设计主要围绕 LoRA [20]，而工业级 LLM 更迫切的需求仍是全参数训练。

Rollout：环境。环境天然是有状态的：代码工作区、浏览器页面或游戏状态都会随着动作不断演化。因此，同一条轨迹的所有动作必须路由到同一个持久环境实例上，也就是说系统必须提供会话亲和性。

无状态奖励。奖励工作者读取轨迹并输出标量值，不需要记住过去评估历史，非常适合共享、多租户的 *Reward-as-a-Service*。它可以按需从零扩容到很大规模，从而最大化资源利用率。

Training。训练阶段天然有状态，需要专用 GPU 集群承载。这一阶段需要与 rollout 异步协同，但自身不适合迁移到无服务器模式。

```
1 import rollart.distributed as rdist
2 from rdist.worker import ActorTrainCls, ActorGenCls,
  RewardCls
3 from rdist import ResourceManager as RM
4
5 class MyActorTrain(ActorTrainCls):
6     @rdist.register(mode="execute_all")
7     def compute_gradients(self, input_tensor):
8         ...
9
10 gen_rm=RM({"H800": list(range(0, 8))},
11           {"H20", list(range(8, 32))})
12
13 class HeteroActorGen(ActorGenCls):
14     @rdist.hw_mapping(
15         hw_affinity={"FrozenLake": "H800", "default": "H20"}
16     )
17     def generate(self, input_ids, tag_name="default"):
18         return self.model.process(prompt)
19
20 class ServerlessRewardWorker(RewardCls):
21     @rdist.register_serverless(
22         attribute='reward_proxy',
23         serverless_url='fc://xxx.xxx')
24     def compute_rewards(self, traj: list):
25         prompt = f"Evaluate the trajectory:{traj}"
26         return ray.get(self.reward_proxy(prompt))
```

Listing 1: RollArt 的声明式编程模型。

4 编程与计算模型

本节介绍 RollArt 如何通过一套声明式编程模型与运行时计算模型落实上述设计原则。

4.1 声明式编程模型

在 RollArt 中，worker 是资源占有的最小单位。Listing 1 展示了其在 worker 函数层面的编程模型，这让运行时能够以细粒度感知资源使用方式。

单控制器模型。工业界很多 RL 框架都采用单控制器模型 [45, 61]，因为它简化了 agentic RL 流水线的搭建。RollArt 也采用这一范式，并用 register 装饰器实现。当执行模式设置为 execute_all 时，运行时会把输入广播给所有 trainer worker，并在各 worker 上调用 compute_gradients。分布式计算与通信由系统负责，用户只需要在执行函数内部实现计算逻辑，再通过不同类型 worker 的函数组合整个训练流程。

硬件亲和和映射（原则 1）。RollArt 通过字典式资源描述支持异构资源组配置。以 HeteroActorGen.generate 为例，用户可借助 hw_mapping 装饰器为某个 worker 方法声明细粒度硬件映射：被标记为 FrozenLake 的生成请求优先路由到 H800，其余请求优先进入 H20。函数参数 tag_name 作为动态路由提示，使每次生成请求都能自动分配到绑定了合适硬件的 worker。这样，每条轨迹都能以最匹配的资源形态执行。

无状态组件注册（原则 3）。用户可以通过 register_serverless 装饰器，把奖励函数等无状态计算绑定到外部 serverless 接口。对用户而言，这与调用本地函数几乎一致；对系统而言，它意味着这部分计算不再占用常驻 GPU，而是可以利用外部平台进行弹性扩缩容。

4.2 计算模型

轨迹级 rollout。RollArt 将单条轨迹视为 rollout 的核心执行单位。每个 EnvManager 维护一条或多条轨迹的状态机：先通过 reset 初始化环境，再进入事件循环，与共享的 LLMPProxy 反复交互。环境将历史观测

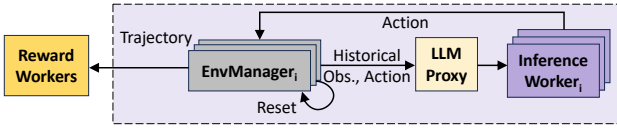


Figure 4: 轨迹级 rollout 调度示意。

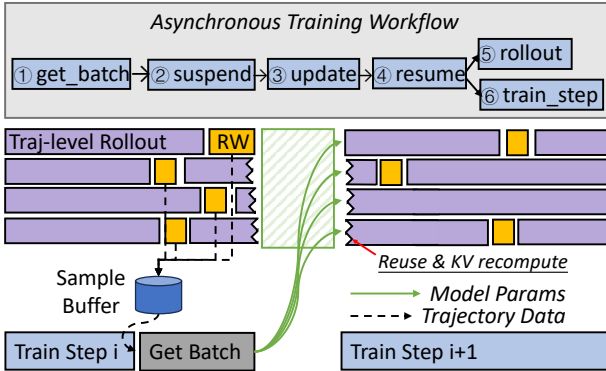


Figure 5: 异步训练 workflow。

与动作序列送入 LLMProxy 生成下一步动作，再把动作应用到环境上，并记录新观测，直到轨迹结束。这种设计让不同轨迹彼此独立，避免批次级同步带来的等待。

冗余环境 rollout。基于轨迹级设计，系统可以执行所谓冗余环境 rollout：即启动多于理论所需数量的环境副本。一旦目标数量的轨迹已经收集完成，其余尚未结束的 rollout 可以直接终止。由于调度以轨迹为粒度，慢环境不会阻塞快环境，从而显著缓解 fail-slow 与 fail-stop 问题。论文实验表明，这一技术可以切实提升 rollout 效率。

异步训练 workflow。Figure 5 展示了 RollArt 如何编排异步训练流程。rollout 阶段中，多个 EnvManager 独立运行，持续生成轨迹并推入 SampleBuffer；推理 worker 与训练 worker 部署在不同 GPU 上，从而形成真正的解耦式训练。

训练阶段会周期性执行权重同步协议：先阻塞式调用 get_batch 从 SampleBuffer 取出一批轨迹，再发出 suspend 暂停采样，随后拉取最新模型并把权重广播给所有推理 worker，最后通过 resume 恢复轨迹级 rollout。上一轮未完成的轨迹也可通过 KV cache 重算在下一轮继续利用。随后系统在已取出的数据上执行 train_step。在异步模式下，training 与 rollout 相互重叠，而系统用异步边界控制轨迹的新鲜度。

异步边界。异步训练允许轨迹在旧版本模型下开始生成、在更新后的新模型下继续执行，因此一批轨迹中可能混杂多个模型版本。样本陈旧性会提高方差并损害稳定性。ARaL [11] 用平均样本新鲜度做约束；RollArt 则引入每轨迹的异步边界 α ，定义为当前 agent LLM 版本与该轨迹启动版本之间允许的最大差值。若模型已推进到版本 n ，则 SampleBuffer 中任何轨迹都必须由不早于 $(n - \alpha)$ 的版本启动；超出该约束的轨迹将被中止。论文实验显示， $\alpha = 1$ 通常能在训练速度和稳定性之间取得较优平衡。

5 系统架构

我们用约 6 万行 Python 从零实现了 RollArt。Figure 6 展示了它的两层结构：分布式运行时层和资源管理器。RollArt 首先解析用户配置，并在分布式运行时之上

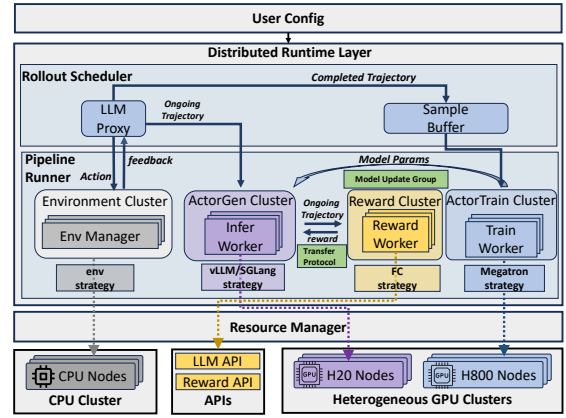


Figure 6: RollArt 的系统架构。

构造训练流水线。运行时包含 rollout 调度器，用于管理 rollout 执行流程；还包含负责分布式计算与通信的 Cluster 运行时。资源管理器则从共享资源池中为各类 worker 分配异构资源。

Rollout Scheduler。调度器通过管理每条轨迹的生命周期来控制 rollout 阶段。LLMProxy 与多个环境管理器交互，并把进行中的轨迹路由到合适的推理 worker。完成的轨迹被加入 SampleBuffer，供后续训练使用。

Pipeline Runner。Pipeline runner 由多个 Cluster 组成，每个 Cluster 在 agentic RL 中承担不同角色，如 reward、environment 或 training。每个集群使用选定执行策略（例如 vLLM [25]、Megatron [46]、Function Compute [3]）组织一组 worker 执行特定类型的分布式计算。Cluster 之间通过 Ray 的 ObjRef [33] 交换轨迹，从而避免频繁复制数据，并以延迟物化方式隐藏传输开销。对于模型权重同步，每个 Cluster 暴露 model_update_group 接口，并结合 NCCL [34] 与 Mooncake [36] 等高吞吐传输引擎，利用 NVLink、InfiniBand 与以太网高效完成同步。

Resource Manager。资源管理器通过持久化元数据存储维护系统关键状态，包括运行时状态、集群资源可用性以及服务 API 的可用性。当收到 worker 部署请求时，它首先查询元数据验证请求，再把所需资源或外部服务绑定到相应 worker 上。

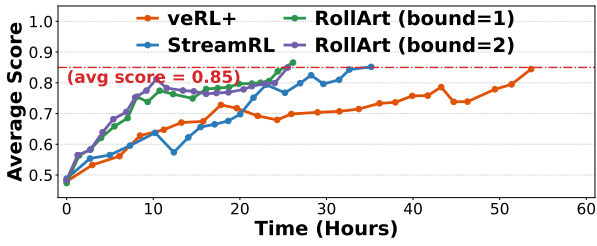
6 性能评测

本节给出端到端评估 (§6.2) 以及对三项设计原则的实验分析。

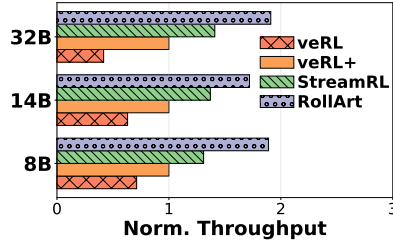
6.1 实验设置

模型与任务。我们在多种 agentic 任务混合数据上训练 Qwen3 [54] 系列模型 (8B–32B)，全部模型的最大上下文长度设为 32K。由于 SWE-bench 难度较高，仅在 Qwen3-32B 上训练该任务。对于数学任务，我们使用一个 7B 奖励 LLM 来验证推理过程。

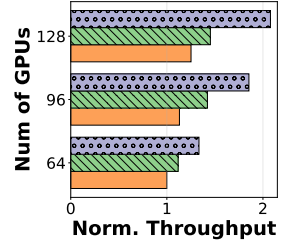
训练配置。我们采用 GRPO [18]，训练 batch size 为 512，group size 为 8，各任务采用均匀采样。异步训练中，我们为 training 固定分配 32 张 H800，并将其余 H20/H800 用于 rollout。对于 rollout，Qwen3-8B、14B、32B 的张量并行度分别设为 1、2、4，并对训练并行方式做调优以最大化吞吐。rollout 使用 vLLM 0.8.4，训练使用 Megatron v0.12.2，并开启 prefix caching 与 CUDA graphs。



(a) 达到目标分数所需时间。



(b) 吞吐效率。



(c) 资源扩展性。

Figure 7: (a) Qwen3-32B 的 time-to-score 对比；(b) 不同方法在不同 LLM 上的归一化吞吐；(c) Qwen3-14B 在不同 H800 GPU 数量下的归一化吞吐。

权重更新引擎。 集群内权重同步使用 NCCL [34] v2.26.5, 跨集群通信使用 Mooncake v0.3.7 [36]。

硬件。 RollArt 部署在一个 96 卡 H800 集群和一个 32 卡 H20 集群上。集群内部通过 400 Gbps InfiniBand 互联, 跨集群通过 200 Gbps 以太网通信。SWE-bench 使用独立 CPU 集群, 其余环境使用另一套 CPU 集群。奖励工作者运行在内部 serverless 平台上。除非特别说明, 实验默认使用 128 张 GPU。

基线。 由于现有开源系统不能完整覆盖我们的 agentic 任务集合, 我们以 veRL 的同步 RL 实现作为主要基线 (veRL), 并进一步为其补上异步奖励计算、异步环境交互与 Reward-as-a-Service, 记为 veRL+。同时, 我们还对比了 StreamRL [58] 及 one-off training 范式 [31]。

6.2 端到端收益

Figure 7 总结了端到端结果。RollArt 在 Qwen3-32B 上显著缩短了达到同等任务分数所需的训练时间, 并在不同模型规模上保持较高吞吐效率。实验表明, 系统在吞吐与 time-to-score 两个维度上都优于同步式与粗粒度解耦式基线。随着 GPU 数量增加, 其扩展曲线仍较平滑, 说明硬件亲和映射和异步编排能有效释放更多资源。

6.3 原则 1: 解耦与硬件亲和的收益

我们对比了单体式部署、仅做阶段级解耦, 以及进一步加入硬件亲和映射三种配置。结果表明, 仅按训练与 rollout 阶段拆分已能带来明显收益, 但对 agentic RL 而言, 这还不足以充分利用异构集群。当系统进一步根据任务特征将 prefill 主导任务路由到 H800、decode 主导任务路由到 H20 时, 吞吐还能继续提升。这一结果验证了原则 1: agentic RL 的异构性不仅存在于阶段之间, 也存在于 rollout 内部。

6.4 原则 2: 异步执行的收益

我们评估了三种细粒度异步机制: 轨迹级环境交互、冗余环境 rollout, 以及异步边界控制。结果显示, 随着 batch size 增大, 轨迹级交互相对于批次级交互的收益从 1.23 倍增长到 2.27 倍, 说明异步环境交互可以在大规模时持续保持高吞吐。对于冗余环境 rollout, 我们在 Qwen3-8B/32K 上同时改变环境组数与 group size, 得到最高 1.62 倍的 rollout 加速, 且两项参数的增加均带来正收益。

对于异步边界, 我们将其从 1 调到 6, 观察不同 LLM 的平均 step time。更大的边界通常降低轨迹因陈旧被中止的概率, 因此可进一步减少 step time; 但收益很

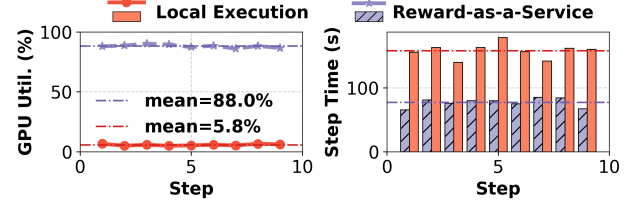


Figure 8: [原则 3]: 本地专用 GPU 与 Reward-as-a-Service 的对比。

快进入平台区, 不同模型的最优值也不相同。综合训练稳定性和最终 time-to-score, RollArt 在实践中通常将边界设为 1。

6.5 原则 3: Serverless 奖励工作者的收益

我们将 Reward-as-a-Service 与本地专用 GPU 奖励部署做对比。在 16 卡集群上并发运行 3 个数学任务作业时, 本地方案需要为奖励 LLM 预留 4 张 GPU, 而 serverless 平台由 3 个作业共享并自动扩缩容。Figure 8 显示, serverless 方案将平均 GPU 利用率从 6% 提高到 88%, 并把单步 rollout 时间从 158 秒降到 77 秒。换言之, 将无状态奖励从本地热备 GPU 中剥离出来, 不仅减少资源浪费, 也让更多 GPU 可以参与 rollout, 从而带来显著端到端收益。

7 RollArt 在生产中的实践

过去六个月里, 已有数千个 agentic RL 作业使用 RollArt 进行后训练。为进一步展示系统的可扩展性与鲁棒性, 我们报告其在一个超过 3000 张 GPU 的生产集群上的部署经验。

工作负载画像 我们使用 RollArt 在内部数学与软件工程数据集上训练了一个百亿级以上参数规模的 MoE LLM。该任务的最大 prompt 长度和回复长度分别为 12k 与 46k, 单任务平均交互轮次从 1 到 48 不等 (见 Figure 9a), 这再次表明 agentic RL 中 prefill 主导与 decode 主导任务会同时存在。大量轮次意味着环境必须极为稳定, 同时 prefill 计算也要足够快。

该作业采用异步训练, 训练 GPU 与生成 GPU 的比例为 1:5。为了兼顾梯度稳定性与 rollout 效率, 我们将异步边界设为 1, 对应的最大单步时长约为 1.5 小时。系统的主要瓶颈来自阻塞式 get_batch 调用: 训练阶段完成 log probability 与梯度计算后, 仍需等待 SampleBuffer 中积累足够多的轨迹。这一等待最高可占迭代时间的 62%, 若能完全消除, 理想情况下端到端训练时间可再下降 22%。

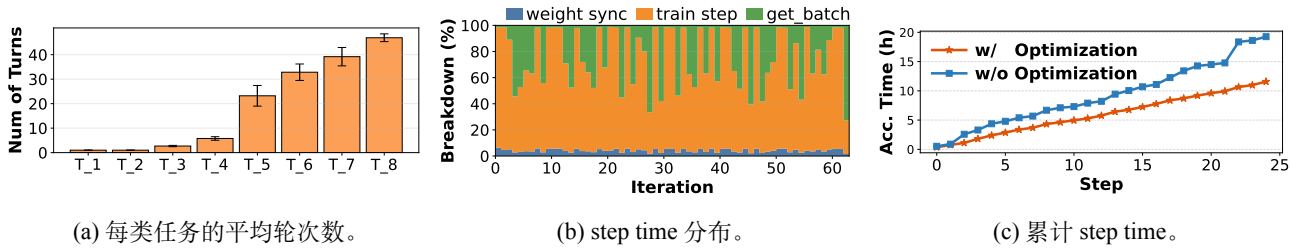


Figure 9: 生产级 agentic RL 工作负载画像。

基于画像的优化根据上述工作负载分析，我们调整了训练与 rollout 的资源比例，并针对 MoE 架构优化了 prefix caching。Figure 9c 表明，这些画像驱动优化在前 25 个 step 内带来了 1.66 倍的端到端加速。除此之外，系统还对环境稳定性与故障恢复进行了专门优化。

环境稳定性优化在 Kubernetes 上同时运行数千个基于 Docker 的 agentic 环境，对稳定性和性能都提出了较高要求。为减少 `env.reset` 中的网络抖动与镜像拉取开销，我们部署了多级缓存架构：第一层为内部镜像仓库镜像源，避免外网访问；第二层为位于计算节点与仓库之间的分布式负载均衡缓存，用于应对大规模并发拉取请求。在 `env.reset` 过程中，客户端优先从缓存获取 Docker 镜像。该方案将环境初始化成功率提升到 99.99% 以上。

系统韧性。我们同时从网络与系统两个层面增强鲁棒性。为减少连接环境时的超时，系统采用基于会话的持久协议而非一次一请求协议，并用指数退避处理临时网络错误。解耦式架构还能隔离故障，使单个环境、奖励 worker 或推理 worker 的问题不至于扩散到其他部分。环境 worker 由 Kubernetes 管理，奖励 worker 由 serverless 平台管理；推理 worker 若失败，首先在原 GPU 上重启，再失败则被剔除，其轨迹从存储引擎恢复并转移到健康 worker；训练 worker 若失败，则从最近检查点重启。实际生产运行一周仅观察到一次故障，证明了这一设计的稳健性。

8 相关工作

RL 后训练系统。大量系统工作都在解决 RL 后训练的系统瓶颈。早期框架 [16, 56, 21] 采用静态、按阶段划分的 GPU 分区，当不同阶段负载动态变化时容易造成整体利用率下降。veRL [45] 则通过混合控制器把多个阶段共置于同一批 GPU 上，以提高硬件效率。随后的一系列系统 [28, 5, 6, 42, 17, 59, 11, 14, 44, 58] 又从投机解码、阶段融合、异步训练、解耦训练等方向提出加速手段。RollArt 延续了解耦和异步的总体路线，但进一步引入以硬件亲和性为中心的细粒度任务映射。

资源解耦。资源解耦最早可追溯到面向内存系统的体系结构工作 [27]，如今已经成为现代系统设计的重要基石。LegoOS [41]、Mira [13] 等系统把硬件拆分成可独立管理的资源池，以提升整体利用率。类似思想在 LLM serving 中尤其常见，例如将 prefill 与 decoding 解耦 [57, 35, 19, 50, 36]，甚至把注意力与 FFN 等子模块进一步拆分 [60, 49]。在 RL 后训练场景中，也有工作把 training 与 rollout 放到不同 GPU 类型上 [58, 52]。相比这些系统，RollArt 提供了一个面向多任务 agentic RL 全生命周期的、更通用也更细粒度的解耦模型。

9 结论

本文设计了一个高效、可扩展的解耦式 RL 训练系统 RollArt。我们围绕三项核心原则构建了其编程模型、计算模型与系统架构：基于硬件亲和性的工作负载映射、细粒度异步执行，以及状态感知计算。微基准与宏基准实验都表明，RollArt 能够在生产级集群中显著提升资源效率、可扩展性与系统鲁棒性。

References

- [1] Introducing openai o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, 2024.
- [2] Alibaba. Rtp-llm. <https://github.com/alibaba/rtp-llm>. Accessed: 2025-12.
- [3] Alibaba Cloud. Alibaba Cloud Function Compute. <https://www.alibabacloud.com/en/product/function-compute>. Accessed: 2025-12.
- [4] Axon-RL. Gem: Generalist environment for multi-task learning.
- [5] Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. Respec: Towards optimizing speculative decoding in reinforcement learning systems, 2025.
- [6] Rongxin Cheng, Kai Zhou, Xingda Wei, Siyuan Liu, Mingcong Han, Mingjing Ai, Yeju Zhou, Baoquan Zhong, Wencong Xiao, Rong Chen, and Haibo Chen. Fast llm post-training via decoupled and best-of-n speculation, 2025.
- [7] Alibaba Cloud. Qwen model repos, 2025.
- [8] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [9] DeepSeek-AI. Deepseek-v3.2-speciale. <https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Speciale>, 2025. Accessed: 2025-12.
- [10] Farama Foundation. Gymnasium - frozenlake environment. https://gymnasium.farama.org/environments/toy_text/frozen_lake/, 2024. Accessed: 2025-09.

- [11] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025.
- [12] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.
- [13] Zhiyuan Guo, Zijian He, and Yiyang Zhang. Mira: A program-behavior-guided far memory system. In *sosp*, 2023.
- [14] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training, 2025.
- [15] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Yicheng Chen, Cunyin Peng, Jinjie GU, and Chenyi Zhuang. Exploring superior function calls via reinforcement learning, 2025.
- [16] Eric Harper, Somshubra Majumdar, Oleksii Kuchaiev, Li Jason, Yang Zhang, Evelina Bakhurina, Vahid Noroozi, Sandeep Subramanian, Koluguri Nithin, Huang Jocelyn, Fei Jia, Jagadeesh Balam, Xuesong Yang, Micha Livne, Yi Dong, Sean Naren, and Boris Ginsburg. NeMo: a toolkit for Conversational AI and Large Language Models, 2025.
- [17] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerrl, 2025.
- [18] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.
- [19] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [20] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [21] Jian Hu, Xibin Wu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.
- [22] Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. Deepretrieval: Hacking real search engines and retrievers with large language models via reinforcement learning. *CoRR*, abs/2503.00223, March 2025.
- [23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.
- [24] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning, 2025.
- [25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [26] Thinking Machines Lab. Tinker, 2025.
- [27] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K Reinhardt, and Thomas F Wenisch. Disaggregated memory for expansion and sharing in blade servers. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*, pages 267–278, 2009.
- [28] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Spec-rl: Accelerating on-policy reinforcement learning via speculative rollouts, 2025.
- [29] Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-rl: Advancing multimodal gui agents from reactive actors to deliberative reasoners, 2025.
- [30] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li. Ui-rl: Enhancing efficient action prediction of gui agents by reinforcement learning, 2025.
- [31] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. Notion Blog.
- [32] Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-rl : A generalist rl-style vision-language action model for gui agents, 2025.
- [33] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework

- for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, Carlsbad, CA, October 2018. USENIX Association.
- [34] NVIDIA Corporation. NVIDIA Collective Communication Library (NCCL). <https://github.com/NVIDIA/ncccl>. Accessed: 2025-12.
 - [35] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *isca*, 2024.
 - [36] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Kimi’s kvcache-centric architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.
 - [37] Alibaba Qoder. Qoder: Agentic coding platform for real software, 2025.
 - [38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [39] ByteDance Seed, Yufeng Yuan, Yu Yue, Mingxuan Wang, Xiaochen Zuo, Jiase Chen, Lin Yan, Wenyuan Xu, Chi Zhang, Xin Liu, et al. Seed-thinking-v1. 5: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*, 2025.
 - [40] SGLang Team. Sglang: Fast serving framework for large language models. <https://github.com/sgl-project/sglang>, 2025. Version 0.4.
 - [41] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *osdi*, 2018.
 - [42] Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpay Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiyang Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. Beat the long tail: Distribution-aware speculative decoding for rl training, 2025.
 - [43] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
 - [44] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous rl post-training framework, 2025.
 - [45] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. verl: Volcano engine reinforcement learning for llm. <https://github.com/volcengine/verl>, 2024.
 - [46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
 - [47] Guijin Son, Hyunwoo Ko, Hoyoung Lee, Yewon Kim, and Seunghyeok Hong. Llm-as-a-judge and reward model: What they can and cannot do, 2024.
 - [48] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning, 2025.
 - [49] StepFun. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding, 2025.
 - [50] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving. In *icml*, 2024.
 - [51] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun

- Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025.
- [52] Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Xiaojiang Zhang, Minglei Zhang, Jiarong Zhang, Wenhao Zhuang, Yuchen Cao, Wankang Bao, Haimo Li, Zheng Lin, Huiming Wang, Haoyang Huang, Zongxian Feng, Zizheng Zhan, Ken Deng, Wen Xiang, Huaixi Tang, Kun Wu, Mengtong Li, Mengfei Xie, Junyi Peng, Haotian Zhang, Bin Chen, and Bing Yu. Seamlessflow: A trainer agent isolation rl framework achieving bubble-free pipelines via tag scheduling, 2025.
- [53] Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [54] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.
- [55] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023.
- [56] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. DeepSpeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales, 2023.
- [57] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024.
- [58] Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation, 2025.
- [59] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. Rlhfuse: Efficient rlhf training for large language models with inter- and intra-stage fusion, 2024.
- [60] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism, 2025.
- [61] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.